

Pay with Quai

The companion to the Frontend UI Handbook. This guide is for engineers wiring **Pay with Quai** into a merchant's product: embedding the checkout UI, registering orders, receiving and **verifying signed webhooks**, and operating on testnet before going live.

Script-tag loader

React components

HMAC webhooks

Order lifecycle

Sandbox / testnet

AUDIENCE	REFS	STATUS	VERSION
Integrating engineers	frontend · backend · contracts	v1 MVP	0.1.0

Contents

- 01 Overview — the two halves of an integration
- 02 Quick start (end-to-end in five minutes)
- 03 Onboarding & credentials
- 04 Embedding the checkout UI
- 05 Checkout modes: modal, hosted page, QR, payment link
- 06 Order lifecycle & on-chain registration
- 07 Webhooks — the source of truth (signing, idempotency, retries)
- 08 Diagnostics & delivery inspection

09 Sandbox & testing on testnet

10 Security & production checklist

01 Overview — the two halves of an integration

Integrating **Pay with Quai** means wiring two sides of your product:

A · Collect payments (customer-facing)

Drop the checkout UI into your site: a **Pay with Quai** button that opens the payment modal, a **QR code** for scanning, or a hosted checkout link. The customer's wallet signs one transaction; funds go **straight to your payout address**. This is UI + a tiny bit of script — the Frontend UI Handbook covers the component internals; this guide covers how you attach them to your page.

B · Settle payments (merchant backend)

Receive **signed webhook notifications** (`payment.confirmed`) on your server, verify the HMAC signature, and finalize orders. This half never touches the browser: it is a **POST** endpoint, one signature check, and idempotent bookkeeping.

What happens per payment

1. Your backend registers an order on-chain (registerOrder — amount, token, expiry)

your site shows payout UI

2. Customer scans / clicks → wallet signs (payOrderNative / approve + payOrder)

one transaction

3. Contract verifies amount, splits fee, pays your wallet, emits **PaymentReceived**

quai zone block

4. Relay waits for finality → **POST payment.confirmed** to your webhook URL

signed, retried, idempotent

5. Your backend verifies the HMAC and fulfils the order

Trust the webhook, not the UI

The checkout UI is convenience. The **only** signal your backend may trust to fulfil orders is a verified `payment.confirmed` webhook (Section 07) — never a browser callback, never an unverified RPC read.

What you need up front

Item	Where it comes from	Used for
Payout wallet address (your merchant address)	Your own Quai wallet, same zone as the deployment	Registered against the backend; funds arrive here

<code>merchantId</code> (e.g. <code>mch_...</code>)	Onboarding response	Referencing your merchant in API calls
<code>webhookSecret</code> (<code>whsec_...</code>)	Onboarding response — shown once	HMAC verification of webhooks
Webhook URL (public https endpoint)	You host it	Where <code>payment.confirmed</code> is delivered
Contract proxy + chain config	<code>contracts/deployments/<network>.json</code>	<code>registerOrder</code> , zone/chain checks

02 Quick start (end-to-end in five minutes)

Every step is covered in detail later; this is the 30,000 ft path.

#	Action	Section
1	Onboard your merchant: <code>POST /v1/merchants</code> (admin bearer) with your payout address, store name and webhook URL → save the <code>webhookSecret</code> from the 201 response.	03
2	Build the webhook receiver: read raw body, verify <code>X-PayWithQuai-Signature</code> , respond <code>200</code> fast, dedupe on <code>event.id</code> .	07
3	Register an order on-chain: <code>registerOrder(orderId, token, amount, expiry)</code> signed by your wallet — do this on a testnet zone first.	06
4	Drop the checkout UI on your page: one script tag, then a button (or QR / payment link) configured with that order.	04–05
5	Pay with a customer wallet → watch the webhook land → fulfil the order. Repeat on mainnet.	09

```
# 1. onboard (bearer = your ADMIN_API_KEY, server-side only)
curl -sX POST localhost:8080/v1/merchants \
  -H "authorization: Bearer $ADMIN_API_KEY" -H 'content-type: application/json' \
  -d '{"address":"0x00..payout","name":"Acme","webhookUrl":"https://acme.test/quai/webhook"'
# → { "merchantId":"mch_...", "webhookSecret":"whsec_...", ... } ← store the secret now
```

03 Onboarding & credentials

POST /v1/merchants — register your payout address

Maps your on-chain payout address → `merchantId` + webhook URL + **webhook secret**.

Field	Rules
<code>address</code>	20-byte hex, checksummed; must be the wallet that receives payouts and live in the same zone as the contract deployment.
<code>name</code>	1–200 chars, display name.
<code>webhookUrl</code>	Must be https and resolve to a public address (SSRF guard). Loopback/private IPs are rejected unless <code>WEBHOOK_ALLOW_INSECURE_URLS=true</code> (local dev only).

- **Response 201:** `{ merchantId, address, name, webhookUrl, active, createdAt, webhookSecret }` — the secret is returned **exactly once**. Store it encrypted in your secrets vault; it can be rotated by re-onboarding (which invalidates the old one), never re-read.
- **409** "merchant already exists" — that address was already onboarded; update it via `PATCH /v1/merchants/:address` (name / webhookUrl / active) which does **not** rotate the secret.
- **Missed payments:** payments that arrived *before* onboarding are queued as `skipped`, not lost — onboarding automatically re-queues them.
- `active:false` pauses delivery (attempts stop, deliveries held); re-activate to resume. Use during deployments/migrations instead of deleting webhooks.

Admin endpoints are server-side only

All `/v1/merchants` and `/v1/deliveries` routes require `Authorization: Bearer <ADMIN_API_KEY>`. Never ship that key to the browser — proxy admin calls through your backend, or run the onboarding from a script/CLI.

Your three operational credentials

Secret	Scope	Handling
<code>ADMIN_API_KEY</code>	Relayer operator (you, the platform)	Environment variable on the backend; never client-side.
<code>webhookSecret</code> (<code>whsec_...</code>)	Per merchant (your webhook verification)	Given once at onboarding; store encrypted; rotating breaks signature checks until updated.
Payout wallet private key	Your merchant wallet	Signs <code>registerOrder</code> ; cold-storage friendly in production. Funds never pass through any intermediate — the contract pays this address directly.

04 Embedding the checkout UI

The checkout components (`PaymentButton`, `PaymentModal`, `PaymentQR`, `PaymentStatus`) — see the Frontend UI Handbook §03–§05) are attached to your page in one of three ways. All three need the same **order config**:

```
{ "merchant": "0x00...", // your payout address (checksummed)
  "orderId": "0x...", // 32-byte hex, unique per order (Section 06)
  "token": "0x0000000000000000000000000000000000000000000000000000000000000000", // 0x0...0 = native QUAID
  "amount": "25000000", // gross, smallest units (18 decimals QUAID, 6 stablecoin)
  "expiry": 1786480000, // optional Unix seconds; 0 = never
  "description": "Invoice #77" } // optional, display only
```

1 • Script-tag loader (vanilla / any framework)

```
<script src="https://cdn.paywithquai.dev/sdk.js"></script>

<div id="checkout-root"></div>
<script>
  const payWithQuai = window.PayWithQuai.init({ chainId: 15000 });
  payWithQuai.attach("#checkout-root", {
    config: { merchant, orderId, token, amount, expiry }, // from your backend
    mode: "modal", // "modal" | "qr" | "button"
  });
</script>
```

The loader renders the trigger on your page (default button, or a QR placeholder) and opens the modal on click. It never touches your DOM elsewhere — it works in a shadow root so your site's CSS cannot break the checkout.

2 • Declarative markup (button auto-upgrade)

```
<button data-pay-with-quai="checkout"
  data-merchant="0x00..." data-order-id="0x..."
  data-token="0x0...0" data-amount="25000000" data-currency="QUAI">
  Pay with Quai — 25 QUAID
</button>
```

Attributes are read once at script load; you can also pass fresh config at click-time via `payWithQuai.openCheckout(config)` — useful for dynamic prices and per-cart amounts.

3 • React (or other bundler)

```
import { PaymentButton, PaymentModal } from '@paywithquai/react';

export function Checkout({ order }) {
  return (
    <>
      <PaymentButton order={order} />
      <PaymentModal order={order}
        onPaid={(ev) => console.log('paid', ev.net)} />
    </>
  );
}
```


05 Checkout modes: modal, hosted page, QR, payment link

`modal` — keep customers on your page (default)

Property	Detail
Flow	Click → overlay with order summary → connect wallet / zone check → approve (ERC-20) + pay → confirmation. States detailed in the UI Handbook §04.
Assets	None — the customer signs from their own wallet. You never handle funds or keys.
ERC-20 note	Stablecoin payments show two wallet steps ("Step 1 of 2 — Allow token"). Native QUA1 is a single step.
Zone guard	Customers whose wallet is on another zone get a blocking warning, not a mysterious failure.

`qr` — scan-to-pay (POS / invoice / printed)

Property	Detail
Payload	The QR encodes this URI (versioned scheme — parse defensively):

```
paywithquai://pay?merchant=0x<checksummed>&orderId=0x<32-byte>&chainId=15000
&token=0x0...0&amount=250000000&description=Invoice%20%2377&expiry=1786480000
```

- The scanning wallet resolves `getOrder(merchant, orderId)` and pays **on-chain truth** — the embedded `amount` is a display hint.
- UI shows a countdown from `expiry`; at zero the QR flips to Expired and a "Regenerate" requires a fresh `registerOrder` (IDs can be reused after `cancelOrder`).
- Fallbacks rendered next to the QR: copy payment link, and deep-link `paywithquai://` for installed wallets.

Hosted checkout page & payment links

- Hosted route (e.g. `https://paywithquai.dev/checkout/0x.../0x...`) renders the full modal as a page — zero integration: just share the URL or wrap it in an iframe from your site.
- Payment link = that URL with the order params embedded; safe to send by chat/email.
- After payment, the page shows live status (it polls the public order endpoint) and optionally `redirect`s to your `successUrl` when the webhook confirms.

Which mode when?

Modal for carts/checkout-pages; QR for invoice screens and physical points-of-sale; payment link for invoices sent to the customer (email/chat); hosted checkout when you cannot add a script (marketplaces, forms, printed receipts).

06 Order lifecycle & on-chain registration

Orders only exist on-chain after **you** register them — the SDK/UI does not create orders. Registration is a plain contract call from your backend (or Checkout-tools UI) using your merchant:

```
await payWithQuai.registerOrder(orderId, token, amount, expiry);
// signed by YOUR wallet · orderId = random 32-byte hex, unique per order
```

Rule	Why it matters
Register before you show any checkout surface	A QR/link for an unregistered order dead-ends at "order not found".
<code>orderId</code> must be unique per order and 32-byte hex	The contract keys orders by <code>(merchant, orderId)</code> ; collisions revert.
<code>amount</code> is gross, in smallest units	The customer pays this exact figure; you receive <code>amount - fee</code> (fee locked at registration).
<code>expiry</code> is optional	<code>0</code> = never expires; you can <code>cancelOrder(orderId)</code> to free it at any time (destructive — confirm twice).
Same zone, always	The contract, your payout wallet and the customer's wallet must all be one zone. Registration from another zone fails.
One payment per order	A settled order cannot be paid twice; the UI shows "already paid" and links to your order page.

created — you called `registerOrder`; `getOrder().settled === false`

customer pays

settled — tx mined; contract split fee (locked bps) → merchant wallet, emitted `PaymentReceived`

finality window + webhook

confirmed — webhook delivered (verified signature, dedupe on id) → fulfil the order

Finality before fulfilment

The relayer only posts `payment.confirmed` after N confirmations and a re-check of `getOrder().settled`. Never fulfil an order from a single unconfirmed RPC read — a shallow reorg could make it disappear. Your "pending" window should tolerate reorgs silently; the webhook is the commit point.

07 Webhooks — the source of truth

On a confirmed payment the relay POSTs this JSON to your `webhookUrl`:

```
{ "id": "0x<txHash>:<logIndex>", "type": "payment.confirmed", "created": 1786480000,
  "data": { "merchantId": "mch_ab12...", "merchant": "0x00...", "orderId": "0x...",
    "payer": "0x00...", "token": "0x0...0", "amount": "25000000",
    "feeBps": 50, "fee": "125000", "net": "24875000",
    "txHash": "0x...", "blockNumber": 12345, "timestamp": 1786479990 } }
```

Field	Meaning / how to use it
<code>id</code>	Idempotency key = <code>txHash:logIndex</code> . Store it; dedupe on it — delivery is at-least-once.
<code>merchantId</code> / <code>merchant</code>	Your id + payout address; reconcile against your records.
<code>orderId</code>	The order you registered — this is your lookup key into your own orders table.
<code>token</code>	<code>0x0...0</code> = native QUA; else the ERC-20 address.
<code>amount</code> (gross) / <code>fee</code> / <code>net</code>	Smallest units, decimal strings. Reconcile against <code>net</code> — that's what hit your wallet. <code>fee = floor(amount × feeBps / 10000)</code> .
<code>txHash</code> / <code>blockNumber</code> / <code>timestamp</code>	For your ledger and explorer links.

Verifying the signature (mandatory)

Every request carries `X-PayWithQuai-Signature: t=<unix>,v1=<hmac>` — HMAC-SHA256 over `${t}.${rawBody}` keyed with your `webhookSecret`. **Verify over the raw bytes, before JSON parsing.** The check is constant-time and rejects timestamps older than 5 minutes (replay protection).

```
import crypto from 'node:crypto';

function verifySignature(secret, header, rawBody, nowSec) {
  const parts = Object.fromEntries(header.split(',').map(p => p.split('=')));
  const r = Math.abs(nowSec - Number(parts.t)); // replay window: ≤ 300s
  if (r > 300) return false;
  const mac = crypto.createHmac('sha256', secret)
    .update(`${parts.t}.${rawBody}`).digest('hex');
  return crypto.timingSafeEqual(Buffer.from(mac), Buffer.from(parts.v1));
}
```

```

async function webhookHandler(req, res) {
  const raw = req.rawBody; // capture BEFORE body-parser JSON
  if (!verifySignature(secret, req.headers['x-paywithquai-signature'], raw, nowSec()))
    return res.status(401).end();
  const ev = JSON.parse(raw);
  if (ev.type !== 'payment.confirmed') return res.status(202).end();
  if (await alreadyProcessed(ev.id)) return res.status(200).end(); // idempotent
  await fulfilOrder(ev.data.orderId, BigInt(ev.data.net)); // net, not amount
  await markProcessed(ev.id);
  res.status(200).end(); // 2xx = delivered, no retry
}

```

Delivery guarantees & handling failures

Property	Guarantee / advice
Delivery	At-least-once. Always dedupe on <code>id</code> ; make your handler idempotent end-to-end (e.g. <code>INSERT ... ON CONFLICT DO NOTHING</code>).
Retries	Non-2xx or timeout → exponential backoff with jitter up to <code>WEBHOOK_MAX_ATTEMPTS</code> , then marked <code>failed</code> (still recoverable, §08). Batches deliver concurrently — one slow endpoint never blocks others.
Timeouts	Respond fast (aim < 5 s). Do heavy work (bank sends, email) on a queue after the 200.
Paused merchants	<code>active:false</code> holds deliveries (no attempts spent); they resume on re-activation.
Redirects	The relayer never follows 3xx — point the URL at the final endpoint.
SSRF	URLs are re-checked by DNS before every delivery; keep them https + public.

08 Diagnostics & delivery inspection

When a payment is late, you have three places to look — in this order:

1 · Public order status (no auth; whoever wants it)

```
GET /v1/orders/:merchant/:orderId
200 { "merchant": "0x00...", "orderId": "0x...", "token": "0x0...0",
      "amount": "25000000", "feeBps": 50, "expiry": "1786480000",
      "settled": true,
      "webhook": { "status": "delivered", "attempts": 1 } }
// settled=false → still payable · 404 → never registered · webhook=null → relayer hasn't
```

Rate-limited (default 60/min/IP) — poll politely (5–10 s), honour `429` + `Retry-After`.

2 · Delivery records (admin)

```
GET /v1/deliveries # recent deliveries: status, attempts, nextAttemptAt, lastErr
POST /v1/deliveries/:id/retry # re-queue a failed delivery as pending (attempts reset)
```

Status	Meaning	Next step
<code>pending</code>	Scheduled / retrying with backoff	Wait (or fix your endpoint); auto-continues
<code>delivered</code>	Your endpoint returned 2xx	Nothing — reconcile your ledger
<code>failed</code>	Retries exhausted after <code>WEBHOOK_MAX_ATTEMPTS</code>	Fix cause → <code>POST</code> <code>/v1/deliveries/:id/retry</code>
<code>skipped</code>	Address had no registered merchant at payment time	Onboard the address — it re-queues automatically

3 · Relay health

```
GET /health → { status, contract, chainId, cursor }
```

If `cursor` is far behind head, indexing is lagging — a relayer config/network problem, not yours. If `status !== ok`, your webhooks can't be produced at all.

Reconcile against net, and against the ledger of webhook ids

Found a payment on-chain but no webhook? Check `skipped` first (address wasn't registered) — the payment is not lost, it replays on onboarding. Once a delivery says `failed`, retry it after fixing the endpoint; do not hand-craft a substitute "confirmation" from explorer data.

09 Sandbox & testing on testnet

Everything above works identically on **Orchard testnet** (`chainId 15000`, RPC `https://orchard.rpc.quai.network`) — the recommended development path.

Testnet environment

Component	Testnet setup
Contracts	<code>cd contracts && npm run deploy:testnet</code> → writes <code>deployments/orchard.json</code> (proxy + mock stablecoin); <code>npm run demo:testnet</code> runs register → approve → pay in one script.
Relayer + API	Point the backend at the testnet zone + proxy address; set <code>START_BLOCK</code> to the deploy block.
Funds	Zone faucet for native QUA; the mock stablecoin <code>mUSDQ</code> has an open <code>mint</code> faucet — anyone can mint test tokens.
Chains / zone	Everything stays in one zone (e.g. Cyprus-1, <code>0x00...</code> addresses) — contract, your payout wallet, test customers.

Local webhook development

```
# sample receiver that verifies signatures and prints events
WEBHOOK_SECRET=whsec_... PORT=9000 npm run webhook-receiver

# local relayers must allow non-https loopback URLs (dev only!)
WEBHOOK_ALLOW_INSECURE_URLS=true # in the relayer's .env
```

Combine with a tunnel (ngrok/cloudflared) to test real https delivery to your backend without deploying.

End-to-end test checklist (before mainnet)

Check	How
Native payment + webhook	Pay a QUA order from a second wallet → verify signed <code>payment.confirmed</code> arrives, net matches <code>amount - fee</code> .
ERC-20 payment	Mint mUSDQ, approve, pay → two-step (approve + pay) flow works.
Idempotency	Your receiver replays the same <code>id</code> → no double fulfilment. Grep your logs for a manual re-delivery test.
Rejection paths	Wrong signature → 401; tampered timestamp → 401; wrong amount order → cannot pay (contract reverts).
Retry & failure	Point your webhook at a 500-ing endpoint → watch backoff → fix → <code>POST /v1/deliveries/:id/retry</code> → delivered.

Expiry	Register with a 1-minute expiry → QR flips to Expired; <code>payOrder</code> after expiry reverts.
Zone mismatch	Connect a wallet on another zone → UI blocks with the zone warning (no broken pay).

10 Security & production checklist

Non-negotiables

#	Requirement
1	Verify every webhook (HMAC, ≤ 5 min freshness) before parsing or trusting it. Default-deny: anything unverified is discarded.
2	Dedupe on <code>id</code> and make fulfilment idempotent — deliveries are at-least-once, retries are expected.
3	Reconcile against <code>net</code> , never <code>amount</code> — that is what your wallet actually received.
4	Fulfil only on the webhook. Non-custodial means no one can move your funds, but a premature "paid" from an RPC read can still ship goods for nothing after a reorg.
5	Webhook URL is https + public ; respond 2xx quickly; no 3xx chaining.
6	Keep zones straight in config: contract, payout wallet and customers must share one zone. Wrong-zone attempts fail loudly in the UI — keep your backend checks just as strict before registering orders.
7	Secrets: <code>webhookSecret</code> and <code>ADMIN_API_KEY</code> live in your secrets vault / env only. The SDK works with zero secrets — if you ever hand a "key" to the browser, you have done it wrong.

Operational checklist (go-live)

- Relayer env: stable `ADMIN_API_KEY`, `PAYWITHQUAI_ADDRESS` = the proxy from deployments JSON, `START_BLOCK` set to deploy block, store on durable disk, one relayer per zone.
- Onboard your production payout address **before** any production payment — payments before onboarding are `skipped` (recoverable, but noisy).
- Verify the webhook receiver is deployed and green before turning on your payment button.
- Test one real payment on mainnet with a small amount through the full loop (register → pay → webhook → fulfil → ledger).
- Set up alerting on `GET /v1/deliveries` for `failed` / `skipped` deliveries and on `GET /health` for relayer lag.
- Keep the fee business rule in mind: merchant views show `amount - fee`; your dashboard and customer receipts must agree with on-chain `feeBps`.

Reading the v1 contract events (advanced / audit)

```
event PaymentReceived(  
    address indexed merchant, bytes32 indexed orderId,  
    address payer, address token,    // address(0) = native QUAI  
    uint256 amount,                  // gross; merchant nets amount - fee  
    uint256 timestamp  
);
```

You can always cross-check any webhook against the chain: filter `PaymentReceived` by `(merchant, orderId)` and verify `getOrder().settled`. The webhook and the event should agree on `amount` / `net` / `blockNumber`.

Pay with Quai · Integration Guide · v0.1.0 · Companion to the Frontend UI Handbook · References: [backend/README.md](#), [contracts/README.md](#), [backend/src/api/server.ts](#), [backend/src/webhooks/signer.ts](#). All amounts are smallest-unit integers on-chain and decimal strings in payloads.